Towards a Decentralized Sphinx Header Construction

Aurélien Chassagne
Université Libre de Bruxelles
Brussels, Belgium
aurelien.chassagne@ulb.be

Iness Ben Guirat
Université Libre de Bruxelles
Brussels, Belgium
iness.ben.guirat@ulb.be

Jan Tobias Mühlberg
Université Libre de Bruxelles
Brussels, Belgium
jan.tobias.muehlberg@ulb.be

Jean-Michel Dricot
Université Libre de Bruxelles
Brussels, Belgium
jean-michel.dricot@ulb.be

Abstract

Sphinx is a packet format commonly used in mixnets and the Lightning Network, designed to prevent adversaries from linking packets across different parts of the network based on their format. A Sphinx packet is constructed by the client and includes encrypted routing information for the message's transmission. While Sphinx provides ease of use, it also allows malicious clients to bias path selection or route packets to unauthorized services. In this paper, we propose a decentralized variant of the Sphinx packet format, distributing header construction across multiple Trusted Third Parties which are assumed to be honest-but-curious, meaning they follow the protocol but may attempt to collude in order to infer routing paths or cryptographic secrets. Our decentralized construction enhances deployment flexibility and paves the way for privacy-preserving mechanisms that could verify the legitimacy of packets at any hop, enabling early rejection of unauthorized traffic without compromising privacy. We implement our scheme in Python and evaluate unlinkability using the NIST SP 800-22 statistical test suite on generated headers. Experimental results suggest that our scheme achieves unlinkability, passing more statistical tests than the original Sphinx format. This work represents a first step towards a decentralized route selection and packet legitimacy enforcement.

CCS Concepts

• **Security and privacy** → *Privacy-preserving protocols*; *Distributed systems security*.

Keywords

mixnet, sphinx, malicious users

ACM Reference Format:

Aurélien Chassagne, Iness Ben Guirat, Jan Tobias Mühlberg, and Jean-Michel Dricot. 2025. Towards a Decentralized Sphinx Header Construction. In *Proceedings of 24th Workshop on Privacy in the Electronic Society (WPES '25).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

WPES '25, Taipei, Taiwan

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Mix network, or mixnet, is an overlay network of servers, called mixnodes, that prevents an adversary from correlating senders with receivers [4, 5, 7, 9, 13, 17, 19, 20]. Various techniques are used in mixnets to resist against a strong adversary who observes all inputs and outputs of the network, typically called a Global Passive Adversary (GPA). First, unlike Tor [11] where all packets sent by a user follow the same path for the entire session (circuit-based), in mixnet each packet follows a different route (packet-based). Second, mixnodes buffer incoming packets with random delays, shuffle and forward them to mitigate timing analysis attacks. Additionally messages are encrypted in layers ensuring that each mixnode only knows its immediate predecessor and successor, and therefore only the final mixnode in the path knows the final destination of a particular message.

Typically, mixnets use the Sphinx packet format, introduced in 2003 by Danezis et al. [8], for encrypting packets. This format provides *bit-wise unlinkability*, meaning an adversary cannot determine whether an output packet corresponds to an input packet based solely on its format. Sphinx is used in various Anonymous Communication Networks (ACNs), including Nym [9], Katzenpost [14], and the Lightning Network [21]. Sphinx packets are constructed by the clients, who encode all the necessary routing information such as the IP addresses and keys. While this simplifies routing, it opens the door for malicious clients to bias the routes and not follow the routing algorithm in order to reduce the overall anonymity provided by the mixnet [12]. Additionally, in systems like Nym, where users must pay a subscription fee to route messages for a specific application, users authenticate using anonymous credentials. However, nothing prevents them from using the network to route traffic for an unauthorized application they have not paid for.

In this paper we present a scheme to create Sphinx headers in a decentralized way based on trusted third parties, while ensuring that these parties learn nothing about the route or destination. Having a decentralized scheme offers greater flexibility for real-world deployment and facilitates ongoing improvements of the technology and its features. We argue that a decentralized approach will simplify the creation of anonymous credentials.
Our work makes the following contributions:

- We propose a decentralized approach to Sphinx header construction;

- We provide a Python implementation of our header construction to support a comprehensive evaluation of our approach in Nym or other mixnets;
- We evaluate the unlinkability property of our decentralized construction by analyzing the randomness of the headers using the NIST SP 800-22 statistical test suite. We show that our construction passes all the tests that the original Sphinx passed. Additionally it succeeds in three tests where the original Sphinx packet format fails.

In the following, we present related work and motivation in Section 2, then we specify and justify our system model in Section 3, where we also describe the considered threat model. We then present our scheme that decentralizes the creation of the Sphinx headers in Section 4 and the evaluation of our proposed solution in Section 5. Finally we conclude and discuss future work in Section 6.

2 Motivation and Related Work

Since Chaum's seminal work on untraceable email in 1981 [4], there has been a great amount of research related to mixnet design [1, 4–7, 13, 15, 16, 20]. However, most systems that have been deployed and used come with their own system on top of the network meaning that they only allow one type of traffic. As stated by Dingledine et al. in [10], "Anonymity Loves Company" meaning that the more messages there are in the network the more privacy the network provides. This idea is further supported by Ben Guirat et al. [2], who show that blending different types of traffic over a mixnet provide better anonymity. For example, consider an instant messaging system where users expect latencies of only a few seconds, and an email application where users tolerate delays of up to a minute. The authors show that mixing these two types of traffic can actually improve the privacy of both types of traffic. This strategy is feasible only in networks like Tor or mixnets, which are designed to support multiple applications, rather than being tied to a single type of traffic.

However, several open research problems remain. For example, how can we prevent certain types of traffic without compromising privacy guarantees ? Such traffic could include clients attempting to reach destinations they have not paid for, or those using the network to access illegal content. Tor mitigates this problem through exit policies, which allow exit nodes to simply drop unwanted traffic. This works in Tor because it is circuit-based meaning that all traffic from a specific user session follows the same path and hence packets are encrypted using symmetric cryptography which is computationally efficient. As a result, dropping traffic at the exit node does not waste significant computation power.

In contrast, mixnets present a more complex challenge. Because each packet follows a different path, mixnets rely on layered public-key cryptography, which is computationally intensive. Dropping packets at the final node, which is typically the only point where service policies can be applied, results in a significant waste of computational resources spent by intermediate nodes on processing and forwarding those packets. Moreover, unlike Tor where relays operate on a voluntary basis, mixnets such as Nym use economic incentives through payment per relayed bandwidth. Consequently, dropping packets at the exit node not only wastes computational resources but also impacts the economic sustainability of the network.

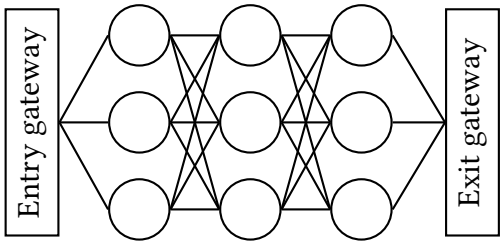

Figure 1: Simplify overview of a mixnet architecture.

This work raises several concrete research questions:

- Could the construction of Sphinx headers be effectively decentralized without compromising core security properties such as unlinkability and integrity ?
- What are the computational and communication overhead introduced by the decentralization ?
- Are those overhead costs preferable to the bandwidth and computational waste caused by malicious users ?
- Does decentralization facilitate the integration of advanced cryptographic mechanisms, such as zero-knowledge proofs for credentials verification ?

3 System and Threat Model

In this section, we present our system model and threat model, and outline the privacy and security requirements our solution should satisfy.

3.1 System Model

A mixnet is a network of servers known as mixnodes, which are arranged in sequential layers, as illustrated in Figure 1. Each packet traverses a fixed, predetermined number of mixnodes. It enters the network via an entry node, is relayed through intermediate nodes in successive layers, and exits through an exit node. Each mixnode reorders and delays packets before forwarding them to the next layer, thereby obfuscating any direct correspondence between input and output. These techniques enhance the mixnet's resistance to a global adversary capable of observing the entire network. For clarity in both formulas and figures, and without loss of generality, we consider a mixnet consisting of three layers (i.e. path of three mixnodes).

This kind of network often relies on privacy oriented packet format of fixed size such as Sphinx [8]. In this work, we will solely focus on the Sphinx packet format. Some privacy-focused companies, like Nym Technologies, commercialize an enhanced mixnet network for services to be integrated with their network for a fee. For example, if Signal is integrated with Nym, clients who want their traffic to be anonymous would route their traffic through the mixnet instead of connecting directly to the Signal server. This prevents adversaries monitoring either the client's device, the server, or any intermediate nodes from correlating sender and recipient. In addition to Sphinx packets, they often improved the system with

extra features such as *anonymous credentials*. For example, each Nym client receives anonymous credentials (based on Coconut credential [22]) to use the mixnet to route their traffic towards specific destinations they have paid for. A potential vulnerability arises when a client obtains a credential for a specific service (e.g. Signal) but uses it to inject traffic directed to an unauthorized or unrelated service. Because the true destination is only revealed at the final hop of the mixnet, earlier mixnodes waste computational resources processing a packet that will ultimately be dropped. This misuse opens the door to denial-of-service (DoS) attacks, where malicious clients flood the system with unauthorized packets that will be detected at the last hop, draining bandwidth and compute capacity.

To address this, we propose a decentralized construction of Sphinx headers involving a set of trusted third parties (TTPs). This decentralized scheme could enable TTPs to update credentials and/or zero-knowledge proofs in accordance with the layered encrypted header, to be verifiable by any mixnodes on the route. As a result, mixnodes could early discard unauthorized traffic before it consumes significant resources. However, the credentials issuance, modification, and verification are outside the scope of this paper and are left for future work.

Figure 2 summarizes our model, which involves three types of entities: the Client, a set of Trusted Third Parties (TTPs), and the Mixnet. The client is considered as potentially malicious. TTPs are assumed to be honest-but-curious, meaning that they correctly follow the protocol but may attempt to collude in order to extract additional information from the data they process. Mixnodes are responsible for enforcing access policies by dropping unauthorized traffic.

3.2 Threat Model

We consider different types of adversaries:

- **Global Passive Adversary (GPA):** An adversary able to observe all traffic entering and leaving the network and each mixnode. The GPA aims to correlate incoming and outgoing packets. Our construction ensures *bitwise unlinkability* of Sphinx packets, such that packet headers appear uniformly random and indistinguishable from one another, thereby preventing correlation.
- **Global Active Adversary (GAA):** An adversary able to modify or inject packets across the entire network. The GAA attempts to alter Sphinx headers in transit or to create a valid Sphinx header. Thanks to the integrity-check mechanisms embedded in the packet format, any such tampering is detected and causes the packet to be discarded by mixnodes.
- **Honest-but-curious TTPs:** In our decentralized header construction scheme, multiple trusted third parties (TTPs) participate in generating the Sphinx headers. These TTPs are assumed to be honest-but-curious, meaning they follow the protocol but may collude in order to extract additional information from the data they process. Our design guarantees that even if a majority of these TTPs collude, but not all of them, they learn nothing about the final destination, the route, or the cryptographic secrets.
- **Malicious Clients:** We consider clients attempting to build Sphinx headers that misrepresent their intended destination by exploiting a valid credential to access a service for which they are not authorized. Such misuse can typically only be detected at the mixnet's exit gateway, where the true destination is revealed and can be compared against the credential. Our decentralized scheme could enable the adaptation of these credentials such that they could be verified at any step in the network.

3.3 Security & Privacy Objectives

Our main security and privacy objectives are the following:

- **Unlinkability:** Our scheme aims to ensure that a global adversary cannot link incoming packets to their corresponding outgoing packets at any mixnode. The original Sphinx packet format introduced by Danezis and Goldberg [8] is known to achieve strong unlinkability guarantees. We target at least the same level of unlinkability, but adopt a relaxed version called *bitwise unlinkability*. If packets appear uniformly random and indistinguishable from one another, we consider that this prevents correlations between incoming and outgoing packets, thereby satisfying the unlinkability requirement.
- **Integrity:** Packets should be protected against tampering. Our scheme should adapt or provide a similar integrity-check mechanism as the original Sphinx, ensuring that any modification in the header is detectable by mixnodes.
- **Collusion Resistance:** Even if a majority of the Trusted Third Parties (TTPs) involved in header construction collude, they must learn nothing about the route, destination or cryptographic secrets. Leakage of routing information could enable timing or metadata correlation attacks across different hops. Therefore, collusion resistance is necessary to maintain strong anonymity and metadata protection in our decentralized scheme.
- **Verifiability:** In the original Sphinx design, credentials are only validated at the exit gateway, once the true destination is revealed. However, credentials should be verifiable at any step in the network, allowing for early detection and discarding of unauthorized packets, reducing wasted computational and bandwidth resources.

4 Our Scheme

In our approach to decentralized Sphinx header construction, each piece of header information is encoded as an elliptic curve point. To achieve this, the encoded string is divided into fixed-size chunks, as illustrated in Figure 3, where each chunk is a single EC point. For example, in the case of a path consisting of three mixnodes, the resulting header contains seven EC points: one destination, three mixnodes and three integrity tags.

Therefore a method to encode and decode information to and from elliptic curve points is required. Specifically, we require a mapping from integers to curve points, noted $\mathrm{Map}()$, and the inverse mapping from curve points to integers, noted $\mathrm{Map}^{-1}()$, such that $\mathrm{Map}^{-1}(\mathrm{Map}(x)) = x$ where x is an integer.

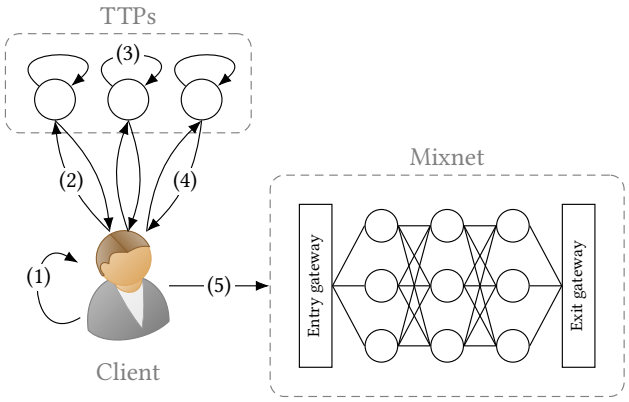

Figure 2: Overview of the decentralized scheme. (1) The client selects a route and generates cryptographic secrets. (2) These are split into shares and sent to TTPs. (3) Each TTP constructs a partial header. (4) The client aggregates them into a final header. (5) The message is sent through the mixnet.

Traditional methods achieve this by directly mapping integers to the x-coordinates of points on the curve. However, this approach could leak information by introducing bias since nearby integers result in nearby points.

To address this, we adopt Elligator [3], which offers stronger privacy guarantees. Unlike the traditional approach, Elligator produces a uniformly distributed output which is computationally indistinguishable from a random curve point. This uniformity is in line with our objectives of preserving anonymity and avoiding linkability.

4.1 Protocol description

The overall decentralized scheme is illustrated in Figure 2. (1) The client first computes a sequence of shared secrets and then splits these secrets, along with the IP addresses of the mixnodes in the path and the final destination, into m shares. (2) Each set of shares is sent to a different TTP, along with the necessary cryptographic element (α). (3) Each TTP independently computes a *partial* Sphinx header using the received shares. (4) The client then aggregates these partial headers to reconstruct the final header, (5) ready for transmission through the mixnet.
In summary, the protocol is divided into four main steps:

- **Setup**: fixes random points as generators (once but should be refreshed);
- **Client**: splits and shares the necessary information to TTPs;
- **TTP**: encrypts the routing information into a partial header;
- **Mixnode**: decrypts and forwards the header.

4.1.1 Setup. In our protocol, routing information is divided into seven chunks, each one encoded into an elliptic curve point (see Figure 3). These points will later be encrypted by adding a masking point of the form $P = sG$, where s is a shared secret and G is the base generator of the curve. However, using the same masking point P for all the seven chunks compromises the *unlinkability* property (see security note in 4.1.3).

To mitigate this, we use a set of seven *independent* generators G_j, one for each chunk. By *independent*, we mean that the scalar relationship between any two generators is unknown. Therefore no one knows the scalars $z_j \in \mathbb{Z}_N$ such that $G_j = z_j G$.

In principle, this setup phase can be executed only once. However, to preserve unlinkability, the set of generators should be refreshed periodically. Every entity (clients, mixnodes, and TTPs) must run the setup phase to ensure they use the same set of generators.

To achieve synchronized and deterministic generators across the system, we derive them from a common seed. This seed is computed as the hash of the current timestamp, truncated to the chosen refresh interval. We simply increment the seed to produce *independent* generators since Elligator already provides a uniform (i.e. pseudo-random) integers to points mapping.

A subtlety with Elligator is that it may map integers to points outside of the base generator's subgroup. To ensure that points remain in the generator subgroup, an extra step is required to *clear the cofactor* by multiplying the resulting point by the cofactor (Curve25519's cofactor is 8).
The final formula for generating the j^{th} chunk's generator is:

$$G_j = 8 \cdot \mathrm{Map}(\mathrm{h}(\text{timestamp}) + j), \quad \text{for } j = 1, \ldots, 7 \tag{1}$$

4.1.2 Client. To send a message to a specific destination, the client first randomly chooses a path through the mixnet and generates a nonce. Using this nonce and the public keys of each mixnode along the path, the client derives a chain of shared secret integers (s_1, s_2, s_3) as follows:

$$(s_1, s_2, s_3) \begin{cases} \alpha_{\mathrm{i}} = x_i\, G \\ S_{\mathrm{i}} = x_i\, K_{\mathrm{i}} \\ s_{\mathrm{i}} = \mathrm{Map}^{-1}(S_{\mathrm{i}}) \\ r_i = \mathrm{h}(\alpha_{\mathrm{i}y} \parallel S_{\mathrm{i}y}) \\ x_{i+1} = x_i\, r_i \pmod n \end{cases} \tag{2}$$

where x_1 is the client's nonce, K_{i} is the public key of the i^{th} mixnode in the path ($K_{\mathrm{i}} = k_{\mathrm{i}} G$), n is the subgroup order (i.e. $(n+1)G = G$), $\parallel$ is the concatenation operator and subscript y refers to the y-coordinate of the point. The scalar r_i acts as a pseudo-randomizer to update the secret x_i for the next hop, ensuring that each cryptographic element α_{i} remains unlinkable across mixnodes.

The IP addresses of the mixnodes and the destination are padded with random bits then mapped to elliptic curve points using Elligator, allowing later recovery of these addresses. Both the resulting IP points and the shared secrets are split into shares and distributed to m different TTPs. In addition, the client sends the hash of each shared secret to these TTPs.

Each TTP then independently computes a partial header from the received shares and returns it to the client. The client aggregates these partial headers by performing chunk-wise point additions to reconstruct the complete encrypted header. Finally, the client sends the header along with α_1 to the first mixnode from the path.

4.1.3 TTP. They receive from the client:

- a share of the destination address (EC point noted Δ);
- a share of each mixnode address in the path (EC points noted N_{i});

- a share of each shared secrets (integers noted s_i);
- the hash of each shared secrets (integers noted σ_i).

The TTP constructs a partial encrypted header by layering the routing information in reverse order along the path, starting from the last mixnode, as illustrated by Figure 3.

At each layer, two new chunks are introduced: the mixnode's address (N_i) and the integrity tag (γ_i). To preserve a fixed-length header while allowing mixnodes to reverse the operation (as in the original design), four additional *filler chunks* are appended. These filler chunks are computed in such a way that at each layer, after applying the corresponding masking points ($s_i G_j$), the last two chunks cancel out and become the identity point (i.e. point at infinity). This mechanism allows these two chunks to be safely truncated at each layer, ensuring a consistent header size while preserving reversibility for mixnodes.

To initialize the header, the TTP encrypts the destination point using the shared secret of the last mixnode and append the required filler chunks. Let Δ be the partial destination point, s_i the partial shared secret corresponding to the i^{th} mixnode in the path, and G_j the j^{th} chunk's generator. The chunks for the last layer (assuming a 3-hop path) are computed as follows:

$$\beta_3 \begin{cases} \beta_{31} = \Delta + s_3 G_1 \\ \beta_{32} = -(s_2 G_4 + s_1 G_6) \\ \beta_{33} = -(s_2 G_5 + s_1 G_7) \\ \beta_{34} = -s_2 G_6 \\ \beta_{35} = -s_2 G_7 \end{cases} \tag{3}$$

Subsequent layers ($i = 2$ and $i = 1$) are computed with Eq. 4 where N_{i+1} represents the elliptic curve point corresponding to the $(i+1)^{\text{th}}$ mixnode in the path and γ_{i+1} is the integrity tag from previous layer.

$$\beta_i \begin{cases} \beta_{i1} = N_{i+1} + s_i G_1 \\ \beta_{i2} = \gamma_{i+1} + s_i G_2 \\ \beta_{i3} = \beta_{(i+1,1)} + s_i G_3 \\ \beta_{i4} = \beta_{(i+1,2)} + s_i G_4 \\ \beta_{i5} = \beta_{(i+1,3)} + s_i G_5 \end{cases} \tag{4}$$

For each layer, the integrity tag is computed with Eq. 5 where $\mathsf{h}^{(j)}(\sigma_i)$ means hashing j times σ_i which is the hash of the i^{th} mixnode's complete shared secret. The integrity tag consists in a pseudo-random weighted sum of the chunks, offset by a secret point derived from the shared secret integer. The pseudo-random weights (i.e. $\mathsf{h}^{(j)}(\sigma_i)$) ensure binding, preventing adversaries from modifying individual chunks. Without these weights, an attacker could add a random point P to one chunk and subtract P from another, thereby preserving the overall sum and bypassing the integrity check. The secret offset (i.e. $s_i G$) prevents curious TTPs from correlating integrity tags. Since TTPs are aware of these pseudo-random weights, omitting this offset would allow them to potentially link some incoming and outgoing mixnode's packets by simply verifying integrity tags.

$$\gamma_i = s_i G + \sum_{j=1}^{5} \left(\mathsf{h}^{(j)}(\sigma_i)\, \beta_{ij}\right) \tag{5}$$

When the first layer is finally computed, it is appended with the integrity tag γ_1 and the first mixnode's point N_1. Finally, the TTP returns the resulting partial header to the client.

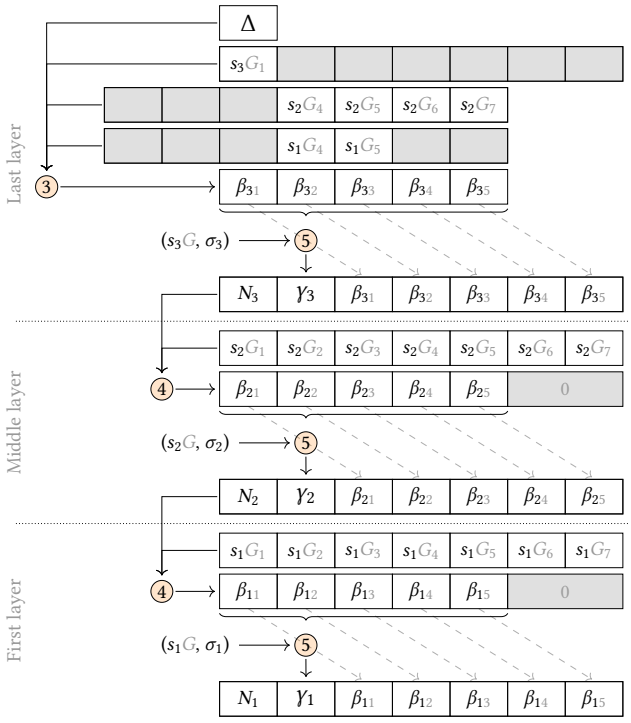

Figure 3: Chunk-wise header encryption done by the TTP. Circled numbers (x) refers to Equation x in the paper.

Security Note. To preserve unlinkability, it is essential that the generators $(G_1, \ldots, G_7)$ are *independent*, meaning their scalar relationships are unknown and computationally infeasible to derive. If the same generator G were used across chunks (or their scalar relationships known), an adversary could compute chunk-wise difference between consecutive layers: $\beta_{ij} - \beta_{i+1j} = s_i G$. Since the shared secret s_i remains consistent within a layer, the resulting differences would reveal a predictable pattern (uniform or preserved scalar relationships). This consistency could allow an adversary to correlate incoming and outgoing packets at a mixnode, thereby breaking the unlinkability property. Using independent generators for each chunk prevents such correlations and is therefore critical to the protocol's security.

4.1.4 Mixnode. When a packet arrives at mixnode i, it begins by extracting the relevant header fields, namely the cryptographic element α_i and the integrity tag γ_i. The mixnode then recomputes the shared secret point S_i using its private key k_i and the cryptographic element α_i:

$$\begin{aligned} S_i &= k_i\, \alpha_i \\ s_i &= \mathsf{Map}^{-1}(S_i) \end{aligned} \tag{6}$$

This shared secret point is subsequently mapped to the shared integer s_i using Elligator inverse mapping. This shared integer is

then used to verify the integrity tag (Eq. 7).

$$\gamma_i \stackrel{?}{=} s_i G + \sum_{j=1}^{5} \left(h^{(j)}(s_i)\, \beta_{ij}\right) \tag{7}$$

If the integrity check passes, the mixnode pads the header by appending two *identity* chunks (i.e. point at infinity) and then updates it as:

$$\beta'_{ij} = \beta_{ij} - s_i G_j \qquad \forall j = 1, \ldots, 7 \tag{8}$$

The first chunk (β'_{i1}), encodes the next mixnode's IP address (N_{i+1}). This point is mapped back to an integer using Elligator and the random padding is removed by keeping the last 128 bits (i.e. size of an IP address).
The second chunk (β'_{i2}) is the next integrity tag (γ_{i+1}) for the remaining five chunks that form the new encrypted routing information (β_{i+1}).

To maintain unlinkability, the cryptographic element α_i must be updated for the next hop. As in the client's step, it is computed using the y-coordinates of both α_i and S_i:

$$\alpha_{i+1} = \text{hash}(\alpha_{i\,y} \parallel S_{i\,y})\ \alpha_i \tag{9}$$

Finally, the mixnode forwards the updated header (α_{i+1}, γ_{i+1}, β_{i+1}) to the next mixnode in the path.

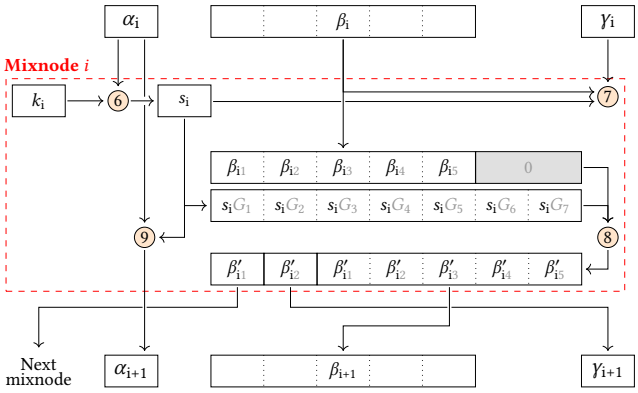

Figure 4: Header processing at mixnode *i*. Circled numbers (x) refers to Equation x in the paper.

5 Evaluation

Our prototype implementation is available on GitHub[1]. The implementation is written in Python 3.13 using the ECPy[2] library for elliptic curve operations. ECPy is well-suited for prototyping due to its simplicity, but it is significantly slower than many EC python libraries (approximately one order of magnitude slower). However, it is one of the few Python libraries that support Montgomery Curve25519, which is essential for our implementation.

We evaluate our system on two main axes: computational complexity and unlinkability (i.e. resistance to correlation between incoming and outgoing packets).

[1]https://github.com/AurelienCha/Decentralized-Sphinx
[2]https://pypi.org/project/ECPy/

Table 1: Number of expensive cryptographic operations per party, where p is the path length (e.g. $p = 3$), and m is the number of TTPs.

	Client	TTP	Mixnode
EC Multiplication	$O(p\,m)$	$O(p^2)$	$O(p)$
EC Addition	$O(p\,m)$	$O(p^2)$	$O(p)$
Point → Integer	$O(p)$	0	2
Integer → Point	$O(p)$	0	0
Hash	$O(p)$	$O(p^2)$	$O(p)$

5.1 Computational Complexity

Instead of measuring raw execution time, which can vary significantly based on hardware and software environments, we focus on the number of expensive cryptographic operations. Table 1 counts these operations per party for m TTPs and path of length p. To compute the total cost for sending a single message, multiply the TTP cost by m and the mixnode cost by p. In typical mixnet implementations, the path includes approximately three to five mixnodes. As a result, $O(p)$ can be considered constant (i.e. $O(1)$).

Consequently, the overall computational burden is more sensitive to the number of TTPs m rather than the path length p. Since our design remains secure as long as a single TTP is honest (i.e. non-colluding), only a small number of TTPs may be required. A deeper analysis of how many TTPs are needed to ensure a desired level of trust remains an open question for future work.

5.2 Unlinkability Evaluation

Quantifying unlinkability remains a fundamental challenge in anonymous communication systems. In Loopix [20], the authors introduce a probabilistic metric called the *expected difference in likelihood* to evaluate *sender-receiver third-party unlinkability*. However, this approach relies on network simulations and specifically assesses the likelihood that a given packet originated from a particular client. In contrast, our focus is on unlinkability at the mixnode level, meaning the difficulty for a Global Passive Adversary (GPA) to correlate an incoming packet with its corresponding outgoing packet through cryptanalysis. Therefore, assuming that greater randomness hinders correlation, we evaluate the statistical randomness of packet headers to approximate this form of unlinkability.

For this purpose, we rely on the NIST SP 800-22 statistical test suite [18], which consists of 15 tests designed to evaluate the quality of random number generators. Each test produces a p-value representing how likely the data could be produced by a uniform source. If the data are truly random, the distribution of p-values across many samples must follow a uniform distribution.

We compare the statistical results (with default parameters) of our implementation with those of the original Sphinx implementation by Danezis[3]. All graphs presented in this section correspond to the results of the NIST statistical test suite applied to 20 distinct datasets. Each dataset is generated under a different setup configuration to assess the variability and robustness of our implementation. To generate a dataset, we simulate a randomized network of 20

[3]https://github.com/UCL-InfoSec/sphinx

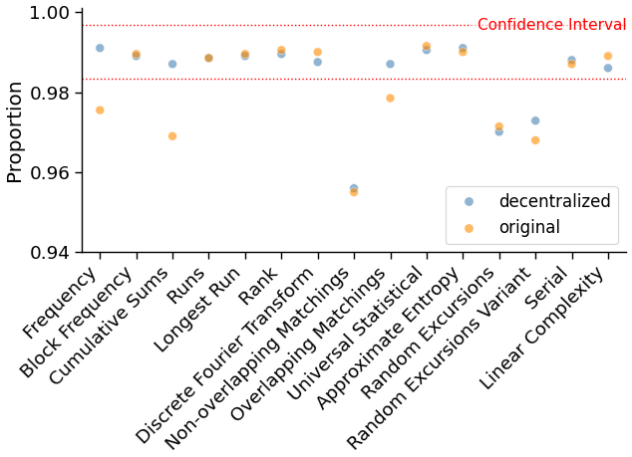

Figure 5: Proportion of successful sequences from our implementation and the original Sphinx implementation over the fifteen NIST SP 800-22 statistical tests (significance level = 0.01).

mixnodes and 3 trusted third parties (TTPs). For each iteration, a random destination IP address and a path consisting of 3 mixnodes are selected. The client then generates shares and distributes them to the three TTPs, which independently compute partial headers. These headers are then aggregated and forwarded through the simulated mixnet. According to the NIST documentation, the statistical test suite requires input sequences of 1 000 000 bits. To meet this requirement, generated headers are concatenated until the total length reaches 1 000 000 bits. Each dataset contains 100 of those concatenated sequences.

NIST outlines two approaches for interpreting the empirical results of its statistical test suite. The first evaluates the proportion of sequences that pass each statistical test, as shown in Fig.5. The second evaluates whether the distribution of p-values is statistically uniform, as would be expected from random sequences. These results are presented in Fig.7.

Figure 5 shows the proportion of sequences that successfully pass each statistical test for both our implementation and the original Sphinx implementation. Following the NIST guidelines, the significance level is set to $\alpha = 0.01$, and the confidence interval is defined as:

$$(1-\alpha) \pm 3\sqrt{\frac{\alpha(1-\alpha)}{m}},$$

where m denotes the number of samples.

Our implementation fails only three tests: *Non-overlapping Matching*, *Random Excursions*, and *Random Excursions Variant*, all of which are also failed by the original Sphinx implementation. In contrast, our implementation successfully passes three tests that the original implementation fails: *Frequency*, *Cumulative Sums*, and *Overlapping Matching*.

Figure 6 presents the distribution of p-values obtained for each test. A Chi-square goodness-of-fit test is applied on each of these histograms to assess their uniformity. The results are shown in Figure 7, with a significance threshold of 0.0001 as recommended

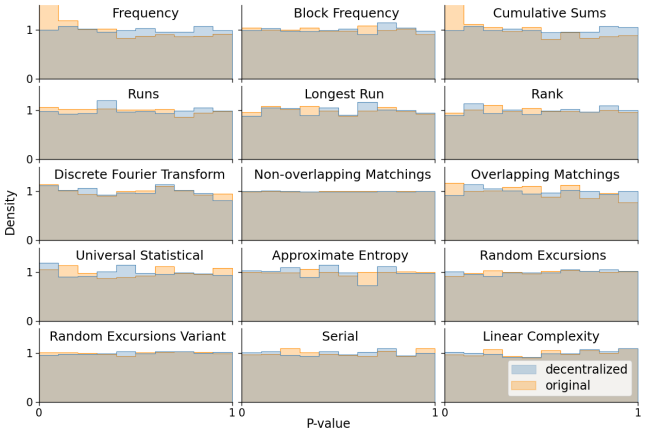

Figure 6: P-value distribution comparison between our implementation and the original Sphinx across the fifteen NIST SP 800-22 statistical tests.

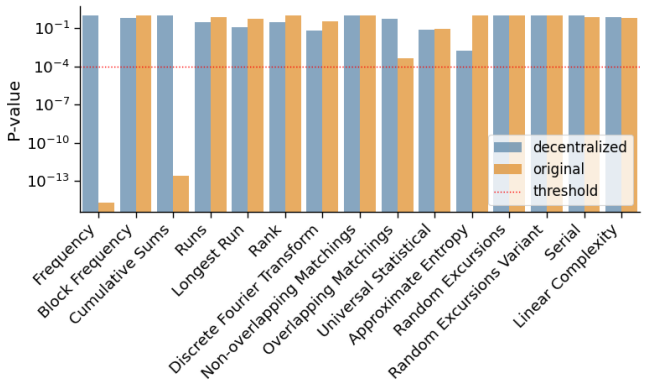

Figure 7: Chi-square test assessing the uniformity of p-value distributions for each dataset across the fifteen NIST SP 800-22 statistical tests.

by the NIST guidelines. Two Chi-square results stand out: the *Frequency* and *Cumulative Sums* tests. Both were successful for our implementation, and failing for the original Sphinx implementation.

To aid interpretation of our results, we briefly describe the specific NIST SP 800-22 tests involved (for further definitions, see the NIST documentation [18]). The *Frequency* test evaluates the proportion of zeros and ones across the entire sequence. It determines whether the number of ones and zeros is approximately equal, as expected in a truly random sequence. The *Matching* tests examine how often predefined m-bit binary patterns appear within the sequence (by default $m = 9$). These tests aim to detect excessive repetitions of specific aperiodic patterns that could indicate some hidden structure. In the *Overlapping* version, a sliding window of m bits is shifted from a single bit at each step, allowing overlapping matches. In the *Non-overlapping* version, the window shifts from m bits in case of matching, avoiding overlapping matches. The *Cumulative Sums* test transforms the binary sequence of 1 and 0 respectively into a sequence of +1 and ˇ1. Then a cumulative sum is

iteratively computed, interpreting the result as a one-dimensional random walk, "randomly" going up (+1) and down (−1). A truly random walk is expected to oscillate around zero. This test assesses the maximum deviation from zero over time and compares it with a truly random walk. The *Random Excursions* tests also rely on the random walk model derived from the *Cumulative Sums* test. The *Random Excursions* test counts the number of times each state has been visited within each cycle[4] and compares this distribution with theoretical expectations for a true random walk. The *Random Excursions Variant* test instead counts the total number of times each state has been visited over the entire walk.

In brief, the *Frequency* evaluates the overall balance between zeros and ones, while the *Cumulative Sums* and *Random Excursions* tests bring additional information about their distribution, and finally the *Matching* tests look for excessive repetition of substrings.

5.3 Discussion

In terms of computational complexity, our implementation is slower than the original Sphinx implementation. However, most of the additional computational workload is distributed among the TTPs, which operate in parallel. Since mixnets inherently trade latency for privacy, this computational overhead may be acceptable in practical deployments.

Regarding unlinkability (i.e. output randomness), our decentralized implementation performs better than the original implementation, as it passes a greater number of statistical tests, as summarized in Table 2. The results suggest that both implementations may exhibit subtle patterns or hidden structure, as indicated by failures in the *Random Excursions* and the *Non-overlapping Matching* tests. This behavior could potentially come from the underlying mathematical framework (e.g. elliptic curves), which is not inherently truly random. Nevertheless, our implementation demonstrates a better overall balance between ones and zeros, as reflected in the *Frequency* and *Cumulative Sums* test results. Given the established security of the original Sphinx protocol, and considering the similarities in core structure, our results indicate that our scheme is a solid starting point toward achieving similar privacy guarantees. However, we emphasize that a thorough formal security analysis remains necessary to fully assess the protocol's security, which we leave for future work.

At this stage, our decentralized scheme successfully achieves three key security and privacy objectives as outlined in 3.3. First, it maintains strong unlinkability, comparable to the original Sphinx. Second, integrity protection is verified at each mixnode, ensuring that message tampering can be reliably detected. Third, our scheme offers robustness against collusion among TTPs, as long as at least one remains honest, preventing them from learning any information about routing information or cryptographic secrets. Since credential issuance, modification, and verification are left for future work, the verifiability of legitimate traffic at each mixnode has not yet been achieved. However, a promising approach involves having TTPs update credentials during header construction, with mixnodes performing further updates during packet processing.

[4] A cycle is a segment of the walk between two successive zero state (i.e. state where the cumulative sum equals 0).

Table 2: Statistical test summary for both implementations (● = Pass, ○ = Fail)

Statistical Test	Decentralized		Original	
	Excessive Rejections	Lacks Uniformity	Excessive Rejections	Lacks Uniformity
Frequency	●	●	○	○
Block Frequency	●	●	●	●
Cumulative Sums	●	●	○	○
Runs	●	●	●	●
Longest Run	●	●	●	●
Rank	●	●	●	●
Discrete Fourier Transform	●	●	●	●
Non-overlapping Matchings	○	●	○	●
Overlapping Matchings	●	●	○	●
Universal Statistical	●	●	●	●
Approximate Entropy	●	●	●	●
Random Excursions	○	●	○	●
Random Excursions Variant	○	●	○	●
Serial	●	●	●	●
Linear Complexity	●	●	●	●

While our scheme introduces higher computational complexity compared to the original Sphinx, it maintains a reasonable polynomial bound, specifically a quadratic complexity. This makes it viable for real-world applications, especially considering the additional features it could enable. The decentralized design could simplify future implementation of advanced features, such as a mechanism to verify the legitimacy of traffic at any mixnode.

6 Conclusion

We have proposed a decentralized scheme that delegates the construction of Sphinx headers to a set of Trusted Third Parties (TTPs). This design ensures that TTPs cannot infer any information about the route or cryptographic secrets, even when colluding, provided that at least one TTP remains honest. To evaluate our approach, we developed a Python implementation. We then performed statistical randomness tests to assess the unlinkability of our Sphinx packets. The results demonstrate that our decentralized construction achieves output randomness comparable to, and in some cases better than, the original centralized Sphinx design. While our Python implementation does not exhibit the performance characteristics required for integration with Nym or any other real mixnet service, it can serve as a template and proof-of-concept for such an integration.

Decentralizing the construction process introduces greater flexibility for real-world deployment and long-term evolution of the protocol. In particular, it opens promising avenues for mechanisms that enable packet legitimacy verification at any hop. Such functionality would enable mixnodes to detect and discard unauthorized traffic early in the path, thereby conserving computational resources and mitigating denial-of-service threats. A potential approach involves TTPs dynamically updating credentials during header construction and mixnodes further updating them during packet processing. We highlight the development of such legitimacy verification mechanisms as an important and promising direction for future research.

Another step toward a fully decentralized scheme would involve delegating route selection to the TTPs themselves. This could help prevent biased or predictable path choices by clients, thereby enhancing overall anonymity.

Before real-world deployment, a comprehensive security evaluation of our scheme is required. This includes formal verification of the protocol's correctness and privacy properties, as well as a detailed threat analysis to identify potential vulnerabilities in our scheme. A careful evaluation of the computational overhead introduced by our approach, compared to the cost of processing illegitimate traffic, will help clarify the overall trade-off of our design.

In summary, our decentralized design offers a promising advancement towards scalable and privacy-preserving anonymous communication. By redistributing trust from a single client to a set of collaborative TTPs, our approach enhances flexibility and robustness of anonymity systems. We view this work as a crucial step toward decentralized cryptographic architectures and hope it contributes to the development of more secure and privacy-preserving communication systems.

Acknowledgments

This research is funded by Belgium Walloon region CyberExcellence program (grant #2110186)

References

[1] Nikolaos Alexopoulos, Aggelos Kiayias, Riivo Talviste, and Thomas Zacharias. 2017. MCMix: Anonymous Messaging via Secure Multiparty Computation. In *26th {USENIX} Security Symposium ({USENIX} Security 17)*. 1217–1234. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/alexopoulos

[2] Iness BEN GUIRAT, Debajyoti Das, and Claudia Diaz. 2023. Blending different latency traffic with beta mixing. *Proceedings on Privacy Enhancing Technologies* (2023).

[3] Daniel J Bernstein, Mike Hamburg, Anna Krasnova, and Tanja Lange. 2013. Elligator: elliptic-curve points indistinguishable from uniform random strings. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 967–980.

[4] David Chaum. 1981. Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms. *Commun. ACM* 24, 2 (1981), 84–88. doi:10.1145/358549.358563

[5] David Chaum, Farid Javani, Aniket Kate, Anna Krasnova, Joeri Ruiter, Alan T Sherman, and Debajyoti Das. 2016. *cMix: Anonymization by high-performance scalable mixing*. Technical Report. Technical report. http://www.cs.bham.ac.uk/~deruitej/papers/cmix.pdf

[6] Lance Cottrell. 1995. Mixmaster and remailer attacks. https://www.obscura.com/~loki/remailer-essay.html

[7] George Danezis, Roger Dingledine, and Nick Mathewson. 2003. Mixminion: Design of a Type III Anonymous Remailer Protocol. In *Proceedings of the 2003 IEEE Symposium on Security and Privacy*. IEEE, 2–15.

[8] George Danezis and Ian Goldberg. 2009. Sphinx: A Compact and Provably Secure Mix Format. In *2009 30th IEEE Symposium on Security and Privacy*. 269–282. http://research.microsoft.com/en-us/um/people/gdane/papers/sphinx-eprint.pdf

[9] Claudia Diaz, Harry Halpin, and Aggelos Kiayias. 2021. The Nym Network. https://nymtech.net/nym-whitepaper.pdf, 38 pages.

[10] Roger Dingledine and Nick Mathewson. 2006. Anonymity Loves Company: Usability and the Network Effect.. In *WEIS*.

[11] David M. Goldschlag, Michael G. Reed, and Paul F. Syverson. 1996. Hiding Routing Information. In *Proceedings of the First International Workshop on Information Hiding*. Springer-Verlag, Berlin, Heidelberg, 137–150.

[12] Iness Ben Guirat and Claudia Diaz. 2022. Mixnet optimization methods. *Proceedings on Privacy Enhancing Technologies* (2022).

[13] Jelle Van Den Hooff, David Lazar, Matei Zaharia, and Nickolai Zeldovich. 2015. Vuvuzela: Scalable private messaging resistant to traffic analysis. In *Proceedings of the 25th Symposium on Operating Systems Principles*. 137–152. https://people.csail.mit.edu/nickolai/papers/vandenhooff-vuvuzela.pdf

[14] Ewa J Infeld, David Stainton, Leif Ryge, and Threebit Hacker. 2025. Echomix: a Strong Anonymity System with Messaging. *arXiv preprint arXiv:2501.02933* (2025).

[15] Albert Kwon, David Lu, and Srinivas Devadas. 2020. {XRD}: Scalable messaging system with cryptographic privacy. In *17th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 20)*. 759–776.

[16] David Lazar, Yossi Gilad, and Nickolai Zeldovich. 2018. Karaoke: Distributed private messaging immune to passive traffic analysis. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*. 711–725.

[17] Ulf Möller, Lance Cottrell, Peter Palfrader, and Len Sassaman. 2003. Mixmaster Protocol — Version 2. IETF Internet Draft.

[18] NIST. 2010. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. Special Publication SP 800-22. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-22r1a

[19] Sameer Parekh. 1996. Prospects for remailers. *First Monday* 1 (August 1996). Issue 2.

[20] Ania M Piotrowska, Jamie Hayes, Tariq Elahi, Sebastian Meiser, and George Danezis. 2017. The loopix anonymity system. In *26th {USENIX} Security Symposium ({USENIX} Security 17)*. 1199–1216.

[21] Joseph Poon and Thaddeus Dryja. 2016. The bitcoin lightning network: Scalable off-chain instant payments.

[22] Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. 2019. Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019*. The Internet Society.

Received 07 July 2025; revised XX July 2025; accepted XX July 2025